

INTI INTERNATIONAL COLLEGE PENANG

Name: Lee Je Rel
Coventry Student ID: 16929706

Course: 6001CMD -
MACHINE LEARNING

Lecturer Name: VASUKY MOHANAN

Coursework: REPORT (100%)

BACHELOR OF SCIENCE WITH HONOURS IN
COMPUTER SCIENCE 3+0,
IN COLLABORATION WITH COVENTRY UNIVERSITY

Semester: AUGUST 2026

GitHub:

<https://github.com/Alust0/6001-CMD-Machine-Learning>

Word Count: 3697 Words (Chapter 1 - 4)

TABLE OF CONTENTS

TABLE OF CONTENTS	2
TABLE OF FIGURES	3
1 Machine Learning Problem Formulation (20 Marks).....	4
1.1 Problem background.....	4
1.2 Business objective	4
1.4 Dataset characteristics and Feature descriptions	5
1.5 Target variable	6
1.6 Possible challenges within the dataset	6
1.7 Dataset represents:	7
2 Comparative Analysis of Learning Paradigms (30 Marks).....	8
2.1 Learning Mechanisms and label Dependencies	8
2.2 Data Requirements and Scalability	8
2.3 Computational Complexity and Algorithm Suitability	8
2.4 Model Interpretability, Advantages, and Limitations	9
2.5 Paradigm Recommendation and Dataset Justification	9
3 Critical Analysis of Data Quality (20 Marks)	10
3.1 Missing values	10
3.2 Duplicate records.....	11
3.3 Outliers	12
3.4 Feature distributions	13
3.5 Skewness	14
3.6 Correlation among variables	15
3.7 Class imbalance	16
3.8 Noise	17
3.9 Data inconsistency.....	18
4 Data Preprocessing Strategy Evaluation (30 Marks).....	19
4.1 Data cleaning.....	19
4.2 Data transformation	19
4.3 Feature Engineering	20

4.4 Data balancing	20
4.5 Proposed preprocessing workflow	21
5 References	23
6 Appendix	25

TABLE OF FIGURES

Figure 1: Verification of Dataset Dimensions and Structural Feature Schema	5
Figure 2: Missing values in Price.....	10
Figure 3: Duplicate records found.	11
Figure 4: Outliers in numeric features.	12
Figure 5: Outliers in numeric features.	13
Figure 6: Price before and after log transformation.	14
Figure 7: Correlation heatmap.....	15
Figure 8: Category proportions for Type, Method and Regionname.....	16
Figure 9: Number of unique values per feature.	17
Figure 10: Text formatting inconsistencies.	18
Figure 11: Preprocessing pipeline output	21

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AUD	Australian Dollar
AVM	Automated Valuation Model
CBD	Central Business District
IQR	Interquartile Range
PCA	Principal Component Analysis
PI	Passed In (auction)
R ²	Coefficient of Determination
RMSE	Root Mean Squared Error
SMOTE	Synthetic Minority Over-sampling Technique

1 Machine Learning Problem Formulation (20 Marks)

1.1 Problem background

The knowledge of the true value of any property is a matter of great interest to any market participant. The property valuation determines the price of a house for a buyer and seller, sets the limit of creditworthiness for a bank, and calculates the size of property taxes for government (Pagourtzi et al., 2003). The traditional method of valuation is carried out by a qualified valuer and an inspection of the property where it is compared to similar houses sold in the region (Kok, 2017). Inspection of property allows an expert to find out nuances invisible from the spreadsheet. However, there are significant drawbacks in this process: different prices can be proposed by different valuers, the valuer can have a subjective bias, the process is too slow and costly, and no individual can keep track of all property sales in a particular city at a particular time (Mayer, Bourassa and Hoesli, 2019).

In a dynamic metropolitan area like Melbourne, house prices fluctuate according to the business cycle, interest rates, and the changing characteristics of neighborhoods (Phan, 2019). In such conditions, real estate agencies, commercial banks, and websites offering real estate services are forced to use Automated Valuation Models (AVMs). This software determines the value of a property based on built-in artificial intelligence and analyzes the data contained in it about the property, its location, and comparable real estate sales data (Kok, 2017).

1.2 Business objective

The objective of this project is to develop an automated valuation model which will enable faster and more cost-effective property valuations. An estimate of a property's value which is given instantaneously is likely to save lenders to mortgages a great deal of time compared to a manual valuation (Kok, 2017). Additionally, because the model uses a data-driven approach, it is likely to avoid the known bias of human valuers who often 'anchor' on a property's previous valuation or sale price (Kok, 2017). Banks, insurers, pension funds and other investors could also instantly revalue their property holdings if necessary, saving the time and expense of yearly or quarterly revaluations which regulators are gradually forcing on them (Kok, 2017).

1.3 Machine Learning objective

From a machine learning perspective, the goal is to train a multiple linear regression model of the form

$$f_{\vec{w},b}(\vec{x}) = \vec{w} \cdot \vec{x} + b$$

then multiplies each input feature by a learned weight in $\vec{w}$, adds them together, and adds a bias b (6001CMD, Week 5). Each input vector $\vec{x}^{(i)} = [x_1^{(i)}, x_2^{(i)}, \dots, x_n^{(i)}]$ holds the features of one property, such as its rooms, type, distance from the CBD and sale details, while the target $y^{(i)}$ is its actual sale price (Price, in AUD). The parameters $\vec{w}$ and b are trained to minimize the squared error cost function

$$J(\vec{w}, b) = \frac{1}{2m} \sum_{i=1}^m (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})^2$$

where m is the number of training examples (6001CMD, Weeks 1 to 5). The model is evaluated using the coefficient of determination (R^2) and Root Mean Squared Error (RMSE), with a target of $R^2 \geq 0.75$.

1.4 Dataset characteristics and Feature descriptions

```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 from scipy import stats
6 import glob
7
8 # Set aesthetic styling parameters for report figures
9 sns.set_theme(style="whitegrid", palette="muted")
10 plt.rcParams.update({'figure.max_open_warning': 0, 'font.size': 11})
11
12 # Detect the uploaded Melbourne Housing CSV file
13 csv_files = glob.glob('MELBOURNE_HOUSE_PRICES*.csv') + glob.glob('melb_data.csv')
14 file_path = csv_files[0] if csv_files else 'MELBOURNE_HOUSE_PRICES_LESS.csv'
15
16 try:
17     df = pd.read_csv(file_path)
18     print(f"Loaded file: {file_path}")
19     print(f"Total Instances (Rows): {df.shape[0]} | Total Features (Columns): {df.shape[1]}")
20     display(df.head(3))
21 except FileNotFoundError:
22     print(f"Could not automatically find the dataset file. Please make sure '{file_path}' or a similar CSV is uploaded to your Colab session.")
```

```
... Loaded file: MELBOURNE_HOUSE_PRICES_LESS.csv
Total Instances (Rows): 63023 | Total Features (Columns): 13
```

	Suburb	Address	Rooms	Type	Price	Method	SellerG	Date	Postcode	Regionname	Propertycount	Distance	CouncilArea
0	Abbotsford	49 Lithgow St	3	h	1490000.0	S	Jellis	1/04/2017	3067	Northern Metropolitan	4019	3.0	Yarra City Council
1	Abbotsford	59A Turner St	3	h	1220000.0	S	Marshall	1/04/2017	3067	Northern Metropolitan	4019	3.0	Yarra City Council
2	Abbotsford	119B Yarra St	3	h	1420000.0	S	Nelson	1/04/2017	3067	Northern Metropolitan	4019	3.0	Yarra City Council

Figure 1: Verification of Dataset Dimensions and Structural Feature Schema

This analysis uses the MELBOURNE_HOUSE_PRICES_LESS.csv file that includes 63,023 instances and 13 columns. Meeting the minimum requirement of 5,000 instances and 10 features. In the dataset, the columns contain numeric values and categorical data.

Furthermore, different columns measure quantity (counts), continuous variables, dates, and categories using textual labels.

Five columns describe the location, including Suburb (the suburb where the property is located), Postcode, Regionname (the metropolitan region of Melbourne, Australia), CouncilArea (the council area responsible for the region), and Distance (distance to Melbourne's Center Business District in kilometers).

Moreover, Rooms (the number of rooms) and Type (the type of the property h: house, u: unit, or t: townhouse describes the property's characteristic). The columns that record details of sales are Address (the address of the property) and SellerG (the real estate company that participated in the sale). The column Date indicates the date of sale, while Method (indicates the selling method for example, S: indicates a standard sale, PI: passed-in auction). The last two columns of the dataset are Propertycount (the number of properties in the suburb) and Price (the price of the property sold). Thus, this dataset describes the sale price of the property in Melbourne, Australia.

1.5 Target variable

The target variable is the price, which is the final price at which the property was sold in Australian dollars. In the case of supervised learning, the target variable y is the dependent variable that needs to be predicted, and, for each example, it is paired with an input example. In this case, the input example is related to the property's characteristics, and the target variable refers to property's sales price (6001CMD, Week 1). Since the property price is a positive real number ($y^{(i)} > 0$), the observed values in this data set can be considered as correct answers (targets). For supervised learning, an algorithm changes parameters such as $\vec{w}$ and b in the model so that its predictions would approximate the targets (6001CMD, Week 1).

1.6 Possible challenges within the dataset

The file MELBOURNE_HOUSE_PRICES_LESS.csv dataset contains a number of challenges that should be addressed prior to training. Firstly, the dataset contains properties with no target value as they were passed in or pulled out at auction, resulting in a number of rows with no numerical value. These rows should be discarded as they provide no information for training. Secondly, house prices demonstrate a strong right-skew, where most sales are clustered in a narrow range but a few exceptionally priced houses exceed this range considerably. Since squared error cost function penalizes large errors more severely, such outliers will inflate the mean absolute error disproportionately. Thirdly, text fields such as Address and SellerG contain thousands of unique values, which would lead to extremely high dimensionality if encoded as categorical variables.

Such sparsity will lead to poor generalization as the model will tend to memorize the training instances, leading to strong overfitting (Cerdeira and Varoquaux, 2020). Finally, the Suburb, Postcode and Regionname fields are largely redundant and provide approximately the same information, which introduces multicollinearity and makes the interpretation of weights in linear models challenging.

1.7 Dataset represents:

This problem can be represented as a supervised regression problem. All past transactions contain the real value of the property, $y^{(i)}$, which is why the task is supervised: one can fit a model to use a property's attributes to predict its price. It is regression because Price is a continuous number ($y \in \mathbb{R}^+$) rather than a category; regression predicts numbers while classification predicts labels, and house price prediction is a standard regression example (6001CMD, Week 1). Also, it would be unnecessarily complicated and difficult to make the task error-free, if one had to discretise the price range and classify properties into price brackets (Bishop, 2006, James et al., 2021). In a supervised regression problem one would proceed by updating the weights, $\vec{w}$, and bias, b , one by one until the cost function $J(\vec{w}, b)$, was minimized (6001CMD, Weeks 1-3). In contrast, unsupervised machine learning is used to identify clusters or outliers in the data, such as partitioning suburbs into groups according to some similarity measure (6001CMD, Week 1). So applying clustering would not be the solution to the given problem, but the best solution is to take a supervised regression task (Kok, 2017).

2 Comparative Analysis of Learning Paradigms (30 Marks)

2.1 Learning Mechanisms and label Dependencies

Supervised learning learns a mapping from inputs to outputs: the model is given an input feature vector $\vec{x}$ together with its correct output label y (6001CMD, Week 1). During training, the model's predictions are compared with these known outputs, and the difference is measured by a cost function (6001CMD, Week 1, James et al., 2021). Gradient descent then repeatedly updates the weights $\vec{w}$ and bias b to reduce the cost $J(\vec{w}, b)$ until it reaches a minimum (6001CMD, Weeks 2–4). Supervised regression therefore depends completely on labels, because the cost cannot be calculated without the actual sale prices $y^{(i)}$ (Bishop, 2006). Unsupervised learning, by contrast, works on data with no output labels y and instead looks for structure in the features themselves, such as natural groups or clusters (6001CMD, Week 1). Rather than measuring prediction error against known answers, unsupervised methods rely on measures such as the distance between data points. K-Means, for example, minimises the squared distance of each point from its cluster centre (Manning, Raghavan and Schütze, 2008).

2.2 Data Requirements and Scalability

Supervised regression needs labelled training examples $(\vec{x}^{(i)}, y^{(i)})$, where every one of the m examples has a recorded target value (6001CMD, Week 1). In this dataset of 63,023 records, any property without a recorded sale price must therefore be removed before training (Pino, 2018). Supervised linear models scale well to large datasets, because vectorisation allows the calculations for all features to run in parallel rather than one at a time (6001CMD, Week 5), and variants such as mini-batch gradient descent further reduce the cost of each update (6001CMD, Week 3). Unsupervised methods such as K-Means clustering do not need labels, so they could use every record, including those with no sale price (6001CMD, Week 1). K-Means itself scales linearly with the number of records, but hierarchical clustering compares every pair of records, so its cost grows much faster as the dataset gets larger (Manning, Raghavan and Schütze, 2008).

2.3 Computational Complexity and Algorithm Suitability

The computational cost of supervised regression depends on the number of features n , the number of training examples m , and the training method (6001CMD, Week 5). For linear regression trained with gradient descent, each update loops over all m examples for each of the n weights, giving a cost of $\mathcal{O}(m \cdot n)$ per iteration (6001CMD, Week 5). The normal equation is an alternative that solves for $\vec{w}$ and b without iterating, but it only works for linear regression and becomes slow when the number of features is large (6001CMD,

Week 5). Suitable supervised algorithms for this task include multiple linear regression, Ridge and Lasso regression, and decision tree regression (6001CMD, Week 1). For comparison, K-Means clustering costs $\mathcal{O}(i \cdot k \cdot m \cdot n)$, where k is the number of clusters and i the number of iterations, so like gradient descent it grows linearly with the data (Manning, Raghavan and Schütze, 2008). Hierarchical clustering is far more expensive: even efficient implementations cost $\mathcal{O}(m^2 \log m)$, and a naive implementation costs $\mathcal{O}(m^3)$, which becomes impractical for over 60,000 records (Manning, Raghavan and Schütze, 2008). Computational cost alone therefore does not rule out K-Means, the deciding factor is that clustering cannot output a price.

2.4 Model Interpretability, Advantages, and Limitations

Supervised linear regression is easy to interpret, because each weight w_j shows how much feature x_j changes the predicted price when the other features stay the same (James et al., 2021). This transparency is useful for lenders and investors who need to understand how a valuation was reached (Kok, 2017). Its main advantage is that it produces actual price predictions whose accuracy can be measured with regression metrics such as Mean Squared Error and R^2 (6001CMD, Week 1). Its main limitations are that it cannot learn from unlabelled data, and that it can overfit when too many features or high-order polynomial terms are added (James et al., 2021). Unsupervised learning has the advantage of not needing labels, which makes it useful for exploring data, detecting anomalies, and grouping similar suburbs (6001CMD, Week 1). However, it cannot predict an individual property's price, and it has no direct way of measuring prediction accuracy (James et al., 2021).

2.5 Paradigm Recommendation and Dataset Justification

Based on the dataset and the business goal, supervised regression is the recommended learning paradigm. The dataset contains 63,023 records, each with property features and, for completed sales, a recorded sale price, which provides the labels needed for supervised learning (Pino, 2018). The goal is to predict *Price*, which is a continuous value ($y \in \mathbb{R}^+$) rather than a category or cluster (6001CMD, Week 1, Kok, 2017). Clustering could group properties by location, but its objective only measures how close points are to their cluster centres and uses no labels, so it has no way to link property features to price (Manning, Raghavan and Schütze, 2008). Supervised regression is therefore the only paradigm that can learn to minimise prediction error and support an automated valuation model (Kok, 2017).

3 Critical Analysis of Data Quality (20 Marks)

3.1 Missing values

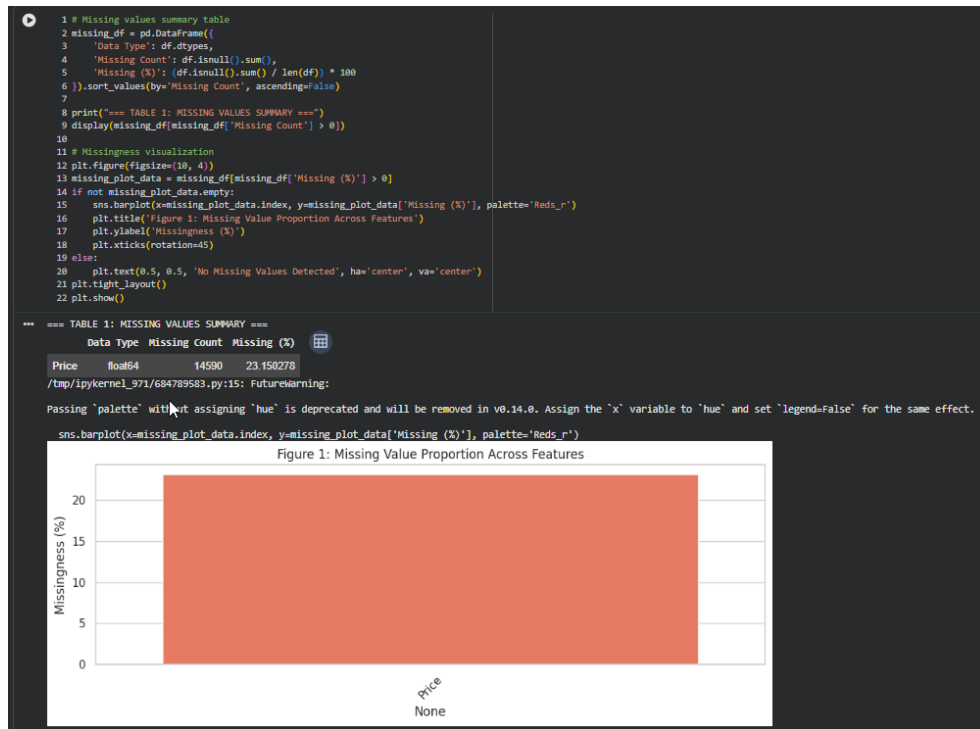


Figure 2: Missing values in Price.

Upon inspection of missing values, it is identified that missing values are only present in the target variable, Price, with 14,590 missing values present which is approximately 23.15% of the total data. All other columns are filled in. These are any properties that do not have a recorded price, such as properties passed in at auction or withdrawn. These rows should be removed prior to training since supervised regression requires that you know the price for each training example.

3.2 Duplicate records

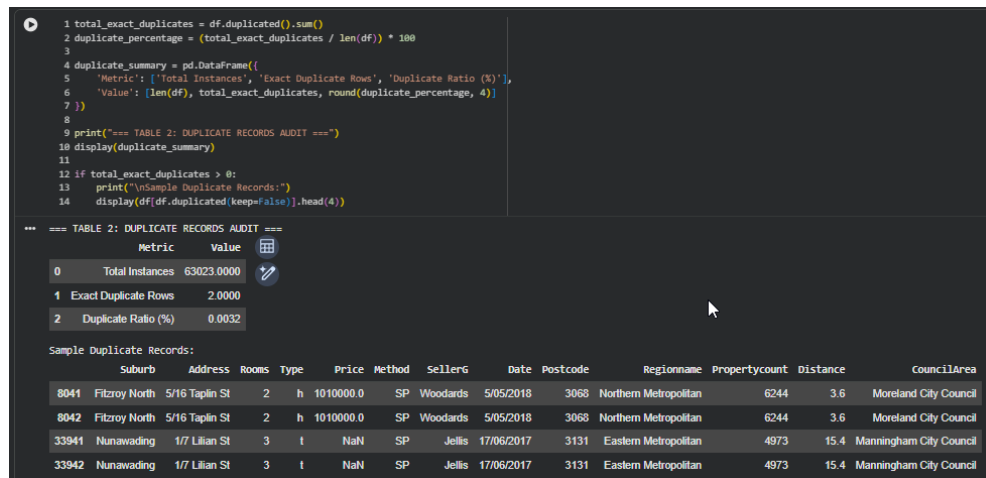


Figure 3: Duplicate records found.

Only 2 of the 63,023 rows are exact duplicates (0.0032%). They are repeated sales and not separate sales, as you can see by checking them. The removal of them does not have meaningful impact on the size of the dataset.

3.3 Outliers

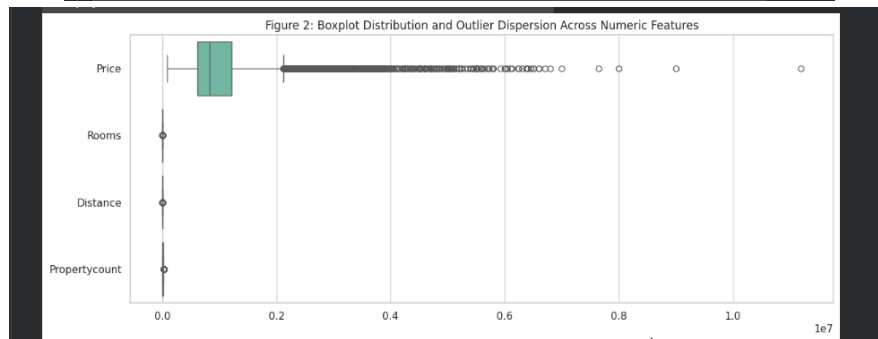
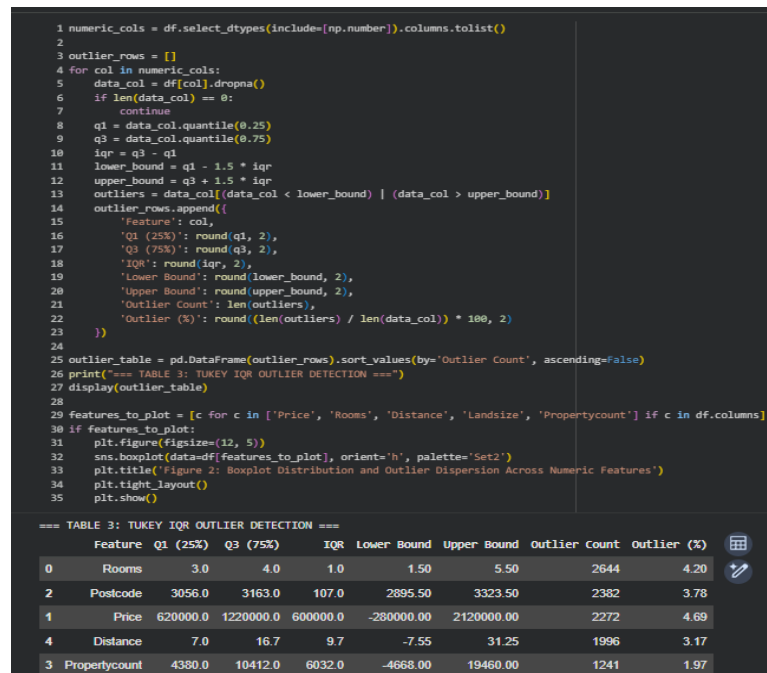


Figure 4: Outliers in numeric features.

The Tukey's Interquartile Range (IQR) method shows that there are 2,272 outliers in the Price data (4.69% of the rows in the table with a recorded price) and 2,644 in the Rooms data (4.20%). It can be seen that prices in the box plot go up to approximately 11 million AUD, well beyond the top limit of 2.12 million AUD.

3.4 Feature distributions

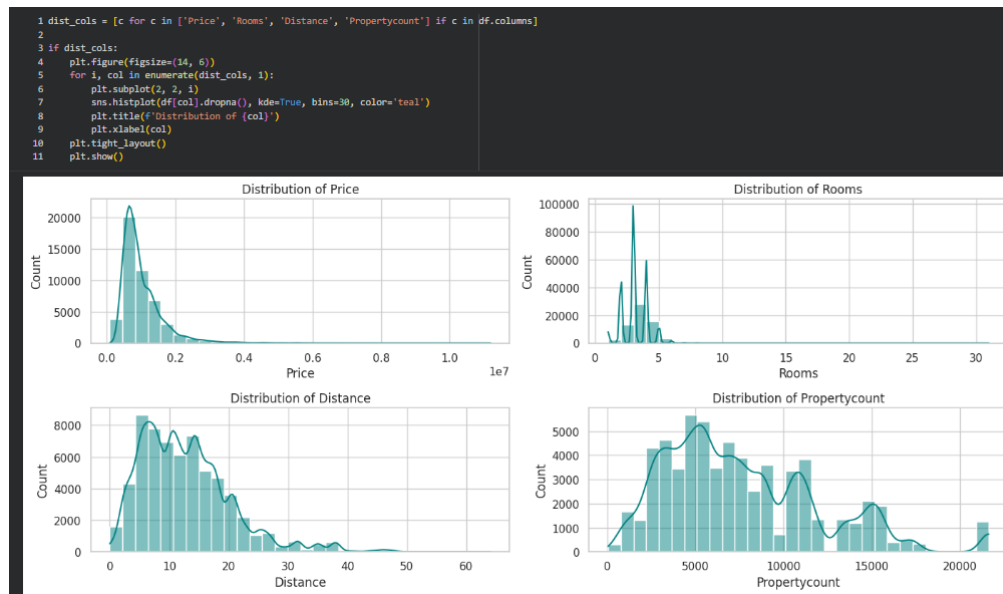


Figure 5: Outliers in numeric features.

Histograms with density curves display the distribution of the numeric features. Price is heavily skewed to the right, with most properties being sold below 2 million AUD, and a long tail of expensive properties. The two other topics (Distance and Propertycount) have several peaks rather than one, and the other two (Rooms and Area) are focused around 3 to 4 rooms.

3.5 Skewness

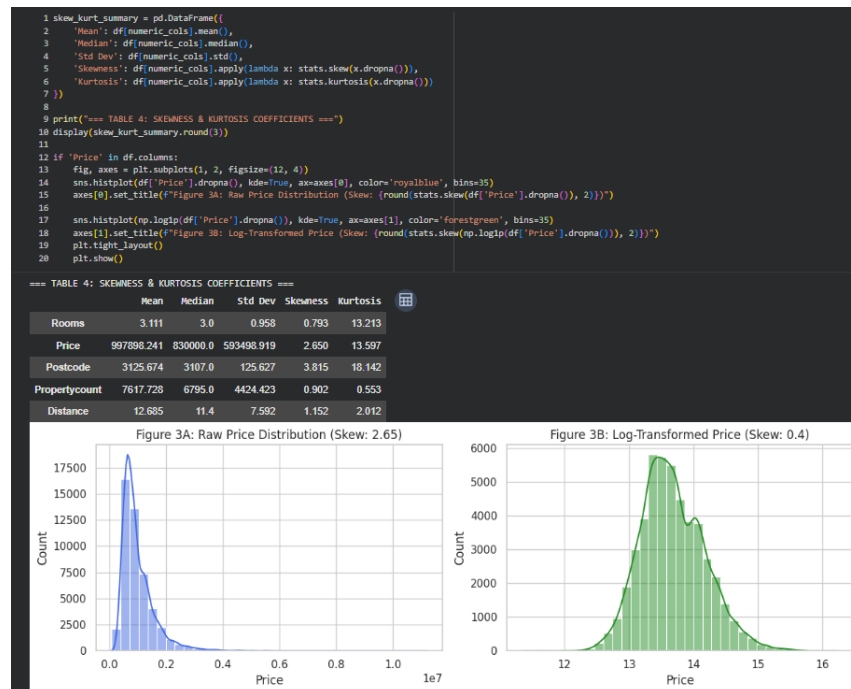


Figure 6: Price before and after log transformation.

A Price distribution is strongly right-skewed (2.650) and has high kurtosis (13.597). The $\log(1+x)$ transformation brings a lognormal shape to the Price distribution, making the long right tail compress and the skewness of the distribution fall from 2.65 to 0.40.

3.6 Correlation among variables



Figure 7: Correlation heatmap.

This heatmap indicates a strong positive correlation between the number of rooms and the price ($r=0.41$), and a negative correlation between the distance of the property from the Melbourne CBD and price ($r=-0.25$), reflecting properties closer to Melbourne CBD see higher selling prices. There is not any strong relationship between any pair of numeric features, so multicollinearity among numeric features is not a significant concern.

3.7 Class imbalance

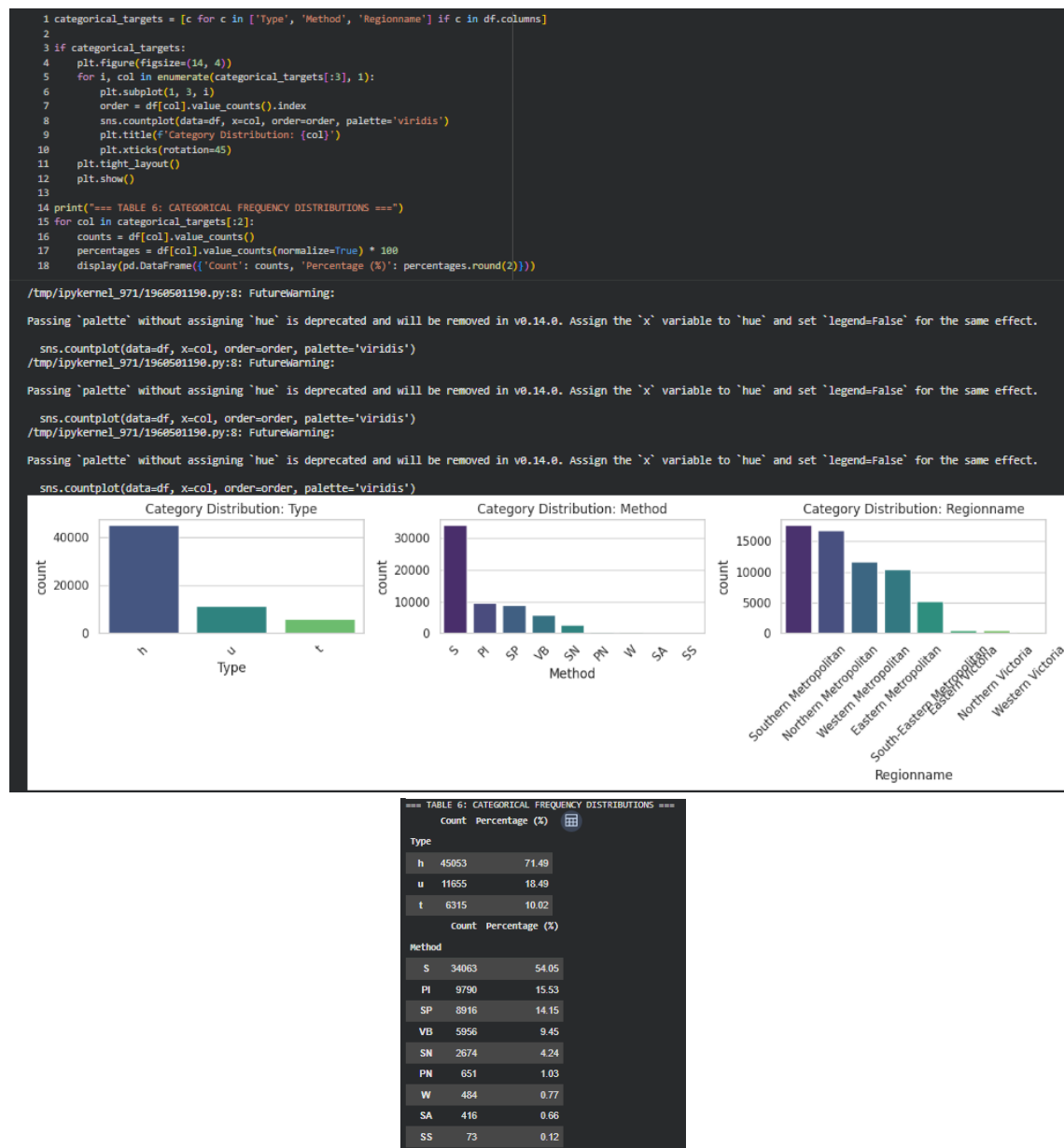


Figure 8: Category proportions for Type, Method and Regionname.

The categorical features are unevenly distributed. For Type, houses (h) make up 71.49% of the dataset, compared with 18.49% for units (u) and 10.02% for townhouses (t). The regional Victoria areas are similarly under-represented compared to Southern or Northern Metropolitan regions with Method and Regionname. This will mean that the model will have much fewer examples of the less common categories and will be less likely to predict the less common categories.

3.8 Noise

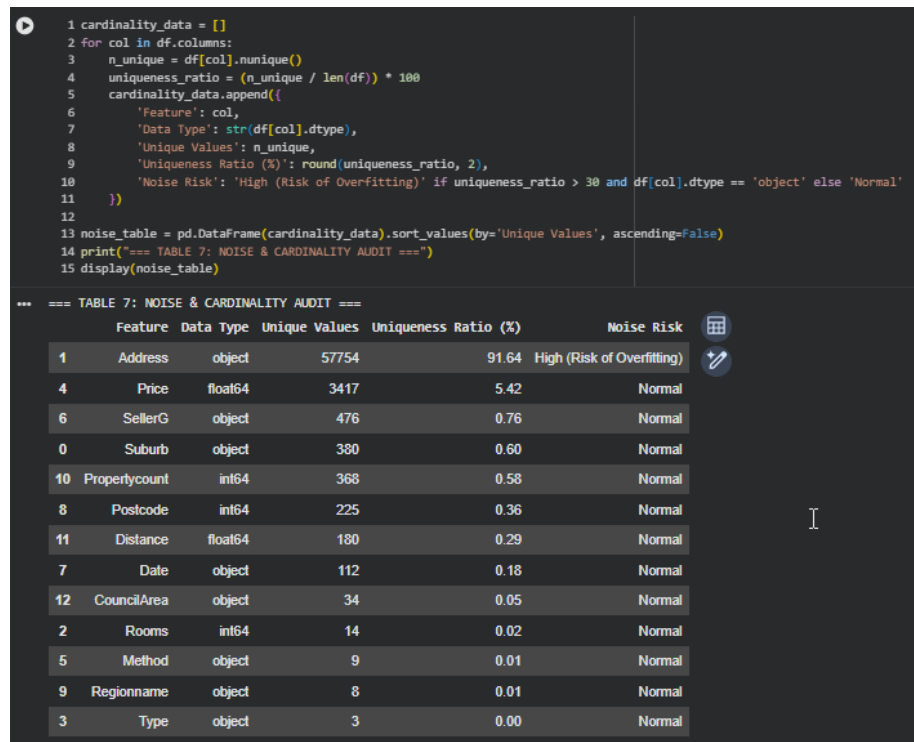


Figure 9: Number of unique values per feature.

The distribution of address has 63,023 occurrences of distinct values (91.64%) which means that almost all rows have a different address. It would result in tens of thousands of empty columns and the model would end up memorising the properties instead of learning the general pattern (overfitting). The best course of action, therefore, is to remove it or summarise it in pre-processing.

3.9 Data inconsistency

```
1 string_cols = df.select_dtypes(include=['object']).columns.tolist()
2
3 inconsistency_audit = []
4 for col in string_cols:
5     raw_unique = df[col].dropna().nunique()
6     stripped_lower_unique = df[col].dropna().astype(str).str.strip().str.lower().nunique()
7     discrepancy = raw_unique - stripped_lower_unique
8     inconsistency_audit.append({
9         'Feature': col,
10        'Raw Unique Levels': raw_unique,
11        'Standardised Levels': stripped_lower_unique,
12        'Inconsistency Discrepancies': discrepancy,
13        'Status': 'Formatting Inconsistency' if discrepancy > 0 else 'Uniform / Clean'
14    })
15
16 inconsistency_table = pd.DataFrame(inconsistency_audit)
17 print("=== TABLE 8: CATEGORICAL INCONSISTENCY AUDIT ===")
18 display(inconsistency_table)
```

=== TABLE 8: CATEGORICAL INCONSISTENCY AUDIT ===

	Feature	Raw Unique Levels	Standardised Levels	Inconsistency Discrepancies	Status
0	Suburb	380	377	3	Formatting Inconsistency
1	Address	57754	57731	23	Formatting Inconsistency
2	Type	3	3	0	Uniform / Clean
3	Method	9	9	0	Uniform / Clean
4	SellerG	476	470	6	Formatting Inconsistency
5	Date	112	112	0	Uniform / Clean
6	Regionname	8	8	0	Uniform / Clean
7	CouncilArea	34	34	0	Uniform / Clean

Figure 10: Text formatting inconsistencies.

The number of text formatting inconsistencies were identified as 23 inconsistencies in Address; 6 duplicate agency spellings in SellerG; and 3 in Suburb. These would result in the same value being considered as a different category; processing the text beforehand is necessary for encoding.

4 Data Preprocessing Strategy Evaluation (30 Marks)

4.1 Data cleaning

Data cleaning is used for cleaning data prior to training, in order to remove any missing values, duplicate rows, and noisy columns. It depends on where missing values are found how their contribution is treated. The cost function $J(\vec{w}, b)$ should be dependent on the true price $y^{(i)}$ of each training example (6001CMD, Week 1). If any price is missing, the corresponding row is dropped, as an average would be used to fill it in, which would be incorrect data for the model to learn. The features can have missing numeric values, which may be replaced by the median (as opposed to the mean, which is influenced by the presence of outliers) and missing categories may be replaced by the most common value. There were no missing values in the features in Section 3.1, and so these steps are provided as a precaution to future data. The 2 duplicate rows are deleted, thus avoiding double-counting of sales during training. Lastly, Address is dropped due to it being 91.64% unique: if Address was one-hot encoded, the model would learn features of the individual words instead of patterns. (James et al., 2021) This comes with the disadvantages of losing 63,023 observations to 48,433 observations after dropping rows with missing prices, and that Address loses information about the street level location after dropping it.

4.2 Data transformation

Scaling numeric features to be on a similar scale to allow for efficient convergence by gradient descent (6001CMD, Week 6). Standardization: rescales a feature to have a mean of 0 and a standard deviation of 1:

$$z = \frac{x - \mu}{\sigma}$$

Normalisation rescales it to a fixed range (6001CMD, Week 6), e.g. $[0, 1]$ or $[-1, +1]$. Both, however, make use of the mean, standard deviation, minimum or maximum, which are all highly influenced by extreme or outlying values like prices up to 11 million AUD. The robust scaling is thus selected, which employs the median (Q_2) and interquartile range, thereby being less susceptible to the effect of outliers (scikit-learn, 2024a):

$$x_{\text{scaled}} = \frac{x - Q_2}{IQR}$$

A skewed target will have a log transform:

$$y_{\text{trans}} = \log(1 + y)$$

This will shrink the long tail of the right side and minimize the skewness of Price from 2.650 to 0.400 which means that only a few properties with very high prices are not dominating training. Other power transformations, like Box-Cox or Yeo-Johnson were also

explored, but will involve fitting an additional parameter λ , which is not necessary with the simpler log transformation as the resulting distribution is nearly symmetric. The drawback of strong scaling is that the values are not constrained to a particular range and to make predictions on the log scale, you will have to transform them back into dollars by calculating $e^{\hat{y}} - 1$.

4.3 Feature Engineering

Use of feature engineering to prepare the inputs, for the linear model to use. Feature selection eliminates columns that are repeated or contain little useful signal, as these columns will be given additional weights that reduce the prediction. Postcode is dropped because it contains more unique values than the other location fields (Address and SellerG); Date is dropped as it is a single field; Address and SellerG are dropped because they contain more values than Postcode. The model retains the properties Rooms, Distance, Propertycount, Type, Method and Regionname. Dimensionality reduction techniques like Principal Component Analysis (PCA) are used to transform the features into a more compact set of new features, which are linear combinations of the original features (James et al., 2021). Here it is not used because these new variables are blends of the original features and are no longer associated with real property characteristics and the model is much more difficult to interpret. Finally, the categorical features Type, Method and Regionname are one-hot encoded, each category being assigned to its own 0/1 column, and not assuming any particular order between categories. One hot encoding adds columns to the dataset, but was done first, so the final dataset has 19 columns remaining, instead of the original 20.

4.4 Data balancing

Balancing techniques like SMOTE, random over-sampling and random under-sampling are used for classification problems, where the number of examples in some classes is much less than others (Chawla et al., 2002). The method is to add additional samples of rare classes or to delete samples of common classes. This problem is a regression problem; hence there are no classes to be balanced: Price is a continuous value ($y \in \mathbb{R}^+$). Artificial properties with invented prices also could enable the model to learn relationships that do not occur in the real market. The under-sampling that is evident in Section 3.7 (houses 71.49% of the data) is not a sampling error and would result in the loss of valid sales. Balancing is thus not applied. Rather, the log transformation and robust scaling are applied in Section 4.2 to deal with the uneven distribution of prices.

4.5 Proposed preprocessing workflow

```
1 from sklearn.model_selection import train_test_split
2 from sklearn.pipeline import Pipeline
3 from sklearn.compose import ColumnTransformer
4 from sklearn.impute import SimpleImputer
5 from sklearn.preprocessing import OneHotEncoder, RobustScaler
6
7 # 1. Target Filtering: Drop records with missing targets
8 if 'Price' in df.columns:
9     df_cleaned = df.dropna(subset=['Price']).copy()
10
11 # 2. Feature Pruning: Drop redundant/collinear columns and granular text identifiers
12 drop_cols = ['Address', 'Date', 'Postcode', 'Bedroom2', 'SellerG']
13 df_cleaned = df_cleaned.drop(columns=[c for c in drop_cols if c in df_cleaned.columns])
14
15 # Separate Target and Predictors
16 X = df_cleaned.drop(columns=['Price'])
17 y = np.log1p(df_cleaned['Price'])
18
19 # Categorize remaining features
20 categorical_cols = [c for c in ['Type', 'Method', 'Regionname'] if c in X.columns]
21 numerical_cols = X.select_dtypes(include=[np.number]).columns.tolist()
22
23 # 3. Build Modular Preprocessing Transformers
24 num_pipeline = Pipeline(steps=[
25     ('imputer', SimpleImputer(strategy='median')),
26     ('scaler', RobustScaler())
27 ])
28
29 cat_pipeline = Pipeline(steps=[
30     ('imputer', SimpleImputer(strategy='most_frequent')),
31     ('onehot', OneHotEncoder(handle_unknown='ignore', sparse_output=False))
32 ])
33
34 preprocessor = ColumnTransformer(transformers=[
35     ('num', num_pipeline, numerical_cols),
36     ('cat', cat_pipeline, categorical_cols)
37 ])
38
39 # 4. Train-Test Partitioning
40 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
41
42 X_train_transformed = preprocessor.fit_transform(X_train)
43 X_test_transformed = preprocessor.transform(X_test)
44
45 print("=== PIPELINE EXECUTION SUMMARY ===")
46 print(f"Original Training Features: {X_train.shape[1]} columns")
47 print(f"Processed Training Matrix: {X_train_transformed.shape[0]} rows, {X_train_transformed.shape[1]} columns")
48 print(f"Processed Test Matrix: {X_test_transformed.shape[0]} rows, {X_test_transformed.shape[1]} columns")
49 else:
50     print("Target variable 'Price' not found in dataset. Skipping preprocessing pipeline setup.")

```

```
=== PIPELINE EXECUTION SUMMARY ===
Original Training Features: 8 columns
Processed Training Matrix: 38746 rows, 19 columns
Processed Test Matrix: 9687 rows, 19 columns
```

Figure 11: Preprocessing pipeline output

The data preprocessing steps are applied in a single scikit-learn pipeline to ensure they are applied to all data each time. The order in which the workflow is carried out is:

- 1 Eliminate rows that have no Price and eliminate duplicate rows.
- 2 Enter Drop Address, Date, Postcode and SellerG.
- 3 Separate out the features X from the target and log-transform the target: $y = \log(1 + \text{Price})$.
- 4 Partition the data between a training data set (80%) and a test set (20%) using a fixed random seed.

- 5 Apply a ColumnTransformer to the train set, to numeric features impute median and apply robust scaling and to categorical features impute most-frequent and apply one-hot encoding.
- 6 Transform the test set using the same fitted transformations.

The transformations performed on the training set do not cause loss of information from the test set to the training set, which makes the test accuracy of the model appear higher than it is (scikit-learn, 2024b). The end result of the pipeline is 38,746 training records and 9,687 test records each containing 19 numeric columns for modelling.

(3697 Words)

5 References

- Bishop, C.M. (2006) *Pattern Recognition and Machine Learning*. New York: Springer. Available at: <https://www.microsoft.com/en-us/research/publication/pattern-recognition-and-machine-learning/>
- Cerda, P. and Varoquaux, G. (2020) 'Encoding high-cardinality string categories for machine learning', *IEEE Transactions on Knowledge and Data Engineering*, 34(3), pp. 1164–1176. Available at: <https://doi.org/10.1109/TKDE.2020.2992529>
- Hastie, T., Tibshirani, R. and Friedman, J. (2009) *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2nd edn. New York: Springer. Available at: <https://hastie.su.domains/ElemStatLearn/>
- James, G., Witten, D., Hastie, T. and Tibshirani, R. (2021) *An Introduction to Statistical Learning: with Applications in R*. 2nd edn. New York: Springer. Available at: <https://www.statlearning.com/>
- Kok, N. (2017) 'Big data in real estate: from manual appraisal to automated valuation', *Journal of Portfolio Management*, 43(6), pp. 1–4. Available at: <https://medium.com/geophy-hq/big-data-in-real-estate-from-manual-appraisal-to-automated-valuation-880a4b0c8aa5>
- Mayer, M., Bourassa, S.C. and Hoesli, M. (2019) 'Estimation and updating methods for hedonic valuation', *Journal of European Real Estate Research*, 12(1), pp. 134–150. Available at: <https://doi.org/10.1108/JERER-08-2018-0036>
- Bishop, C.M. (2006) *Pattern Recognition and Machine Learning*. New York: Springer. Available at: <https://www.microsoft.com/en-us/research/publication/pattern-recognition-machine-learning/>
- James, G., Witten, D., Hastie, T. and Tibshirani, R. (2021) *An Introduction to Statistical Learning: With Applications in R*. 2nd edn. New York: Springer. Available at: <https://www.statlearning.com/>
- Kok, N. (2017) 'Big data in real estate: From manual appraisal to automated valuation', *Geophy HQ*. Available at: <https://medium.com/geophy-hq/big-data-in-real-estate-from-manual-appraisal-to-automated-valuation-880a4b0c8aa5> (Accessed: 4 October 2026).
- Pino, A. (2018) *Melbourne Housing Market*. Kaggle. Available at: <https://www.kaggle.com/datasets/anthonypino/melbourne-housing-market>

Manning, C.D., Raghavan, P. and Schütze, H. (2008) *Introduction to Information Retrieval*. Cambridge: Cambridge University Press. Available at: <https://nlp.stanford.edu/IR-book/>

Chawla, N.V., Bowyer, K.W., Hall, L.O. and Kegelmeyer, W.P. (2002) 'SMOTE: Synthetic Minority Over-sampling Technique', *Journal of Artificial Intelligence Research*, 16, pp. 321–357. Available at: <https://doi.org/10.1613/jair.953>

James, G., Witten, D., Hastie, T. and Tibshirani, R. (2021) *An Introduction to Statistical Learning: With Applications in R*. 2nd edn. New York: Springer. Available at: <https://www.statlearning.com/>

scikit-learn (2024a) 'RobustScaler'. Available at: <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.RobustScaler.html>

scikit-learn (2024b) 'Common pitfalls and recommended practices'. Available at: https://scikit-learn.org/stable/common_pitfalls.html

6 Appendix



Digital Receipt

This receipt acknowledges that Turnitin received your paper. Below you will find the receipt information regarding your submission.

The first page of your submissions is displayed below.

Submission author: LEE JE REL
Assignment title: Draft Plagiarism Checker
Submission title: no toc and coverpage_LEE JE REL_P22014086_6001CMD_CW1....
File name: no_toc_and_coverpage_LEE_JE_REL_P22014086_6001CMD_CW...
File size: 987.54K
Page count: 22
Word count: 4,382
Character count: 24,230
Submission date: 04-Oct-2026 10:02PM (UTC+0800)
Submission ID: 3049777897

4.4 Data balancing	20
4.5 Proposed preprocessing workflow	21
References	23
Appendix	25

TABLE OF FIGURES

Figure 1: Verification of Dataset Dimensions and Structural Feature Schema	5
Figure 2: Missing values in Price	10
Figure 3: Duplicate records found	11
Figure 4: Outliers in numeric features	12
Figure 5: Outliers in numeric features	13
Figure 6: Price before and after log transformation	14
Figure 7: Correlation heatmap	15
Figure 8: Category proportions for Type, Method and Regionname	16
Figure 9: Number of unique values per feature	17
Figure 10: Text formatting inconsistencies	18
Figure 11: Preprocessing pipeline output	21

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AUD	Australian Dollar
AVM	Automated Valuation Model
CBD	Central Business District
IDR	Interquartile Range
PCA	Principal Component Analysis
PI	Passed in Auction
R ²	Coefficient of Determination
RMSE	Root Mean Squared Error
SMOTE	Synthetic Minority Over-sampling Technique

ORIGINALITY REPORT

5%

SIMILARITY INDEX

5%

INTERNET SOURCES

1%

PUBLICATIONS

1%

STUDENT PAPERS

PRIMARY SOURCES

1	amjadizhar.blog Internet Source	1%
2	espace.etsmtl.ca Internet Source	<1%
3	Submitted to Victoria University Student Paper	<1%
4	www.articledge.com Internet Source	<1%
5	www.mdpi.com Internet Source	<1%
6	mdpi-res.com Internet Source	<1%
7	Submitted to Swinburne University of Technology Student Paper	<1%
8	pdfs.semanticscholar.org Internet Source	<1%
9	www.fynd.academy Internet Source	<1%
10	www.researchgate.net Internet Source	<1%
11	docksci.com Internet Source	<1%
12	aemps.ewapub.com Internet Source	<1%
13	wikimili.com Internet Source	<1%
14	Gustavo E. A. P. A. Batista. "A study of the behavior of several methods for	<1%